

Semgrep 自写规则实验报告

基于真实小型仓库的人工源码分析与规则刻画

学号	25140907
姓名	李彦霏
课程	开源软件
实验主题	人工源码分析 + Semgrep 自写规则扫描
扫描目标仓库	code-repo (Flask intrapanel, 7 个 Python 文件, 238 LOC)
规则文件	.semgrep/sqli-string-format.yml
缺陷类别	SQL 注入(字符串格式化拼接 → DB-API execute)
对应 CWE / OWASP	CWE-89 / OWASP A03:2021 — Injection

一、人工源码分析

1.1 通读相关模块

通读 `app.py`、`db.py`、`utils.py` 三份核心模块。其中 `db.py` 是数据访问层(DAL),封装了 `find_user`、`find_user_by_email`、`create_user`、`store_token`、`list_tokens` 五个查询函数,直接操作 `sqlite3` 游标。

Flask 路由层(`app.py`)在 `/users` 与 `/register` 中,把 `request.args.get('q')`、`request.args.get('email')`、`request.form['username']` 等用户输入,未做任何过滤就传给 DAL 函数;DAL 内部再用字符串格式化拼成 SQL,经 `cur.execute(query)` 直接执行。整条数据流从 Source 到 Sink 没有任何转义或参数化处理。

1.2 选定的缺陷类别

SQL 注入(字符串格式化型,CWE-89)。具体表现为:

- % 拼接(C 风格):`db.py:find_user`
- f-string 拼接:`db.py:find_user_by_email`
- `.format()` 拼接:`db.py:create_user`

1.3 为何适合用 Semgrep 规则表达

- 语法可静态识别。三种字符串格式化都是 Python AST 中可枚举的固定形态(`BinOp(%)`、`JoinedStr`、`Call(.format())`),不依赖运行时类型,Semgrep 的语法+元变量已足够覆盖。
- 危险 Sink 收敛。DB-API 把拼接好的字符串送入 `cursor.execute()` / `executemany()`,接收方法名稳定,适合作为定位锚点。
- 判据明确。「query 来自字符串格式化」与「query 进入 execute」两点同时成立即为 True Positive,不需要复杂的过程间分析。
- 可重复性高。不同项目里同样的坏模式反复出现,把判据沉淀成规则后可在 CI 中持续巡查,优于一次性的人工审计。

1.4 「合适触发」的判据

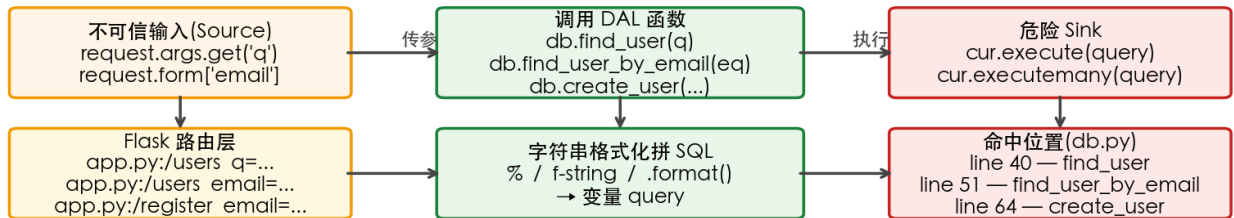
- 判据 1 — Sink 形态:调用形如 `$CUR.execute(...)` 或 `$CUR.executemany(...)`,游标变量名不限。
- 判据 2 — Query 来源:第一参数是 f-string、% 拼接或 `.format()` 调用产生的字符串,可以是 inline,也可以是同一作用域内先赋值后执行。
- 判据 3 — 绑定一致:变量形态下,赋值的左值与 `execute` 的实参必须是同一个变量(由元变量 `$Q` 强制约束)。
- 反例豁免:第一参数为字面量字符串、且第二参数是占位符元组/字典(参数化查询)的调用不应命中。

二、源码级分析流程图

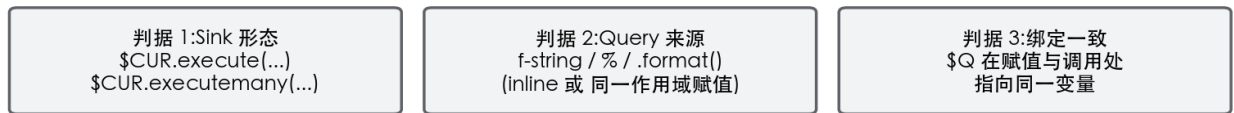
流程图刻画了从不可信输入(Flask 路由读取的用户参数)经过 DAL 函数,在 DAL 内部经字符串格式化拼成 SQL,最终进入 `cur.execute()` 危险 Sink 的完整路径,并在下半部分映射到 Semgrep 规则的三条判据与最小 pattern 片段。

源码级分析流程图 — SQL 注入(字符串拼接 → execute)

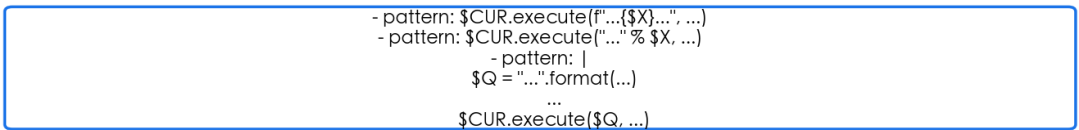
rule: python-sqli-string-format-execute



Semgrep 规则映射



pattern-either(节选)



三、自写 Semgrep 规则

文件路径: .semgrep/sql-string-format.yml

```
rules:
- id: python-sqli-string-format-execute
  languages: [python]
  severity: ERROR
  message: >-
    SQL query is built via string formatting (% , f-string, or .format())
    and then passed to a DB-API execute()/executemany(). This is SQL
    injection. Use parameterized queries: cursor.execute(sql, (param,)).
  metadata:
    category: security
    cwe: "CWE-89: Improper Neutralization of Special Elements used in an SQL Command"
    owasp: "A03:2021 - Injection"
    technology: [python, sql, sqlite, pycopg2, mysql]
    author: "25140907"
    references:
      - https://cwe.mitre.org/data/definitions/89.html
      - https://bandit.readthedocs.io/en/latest/plugins/b608_hardcoded_sql_expressions.html
  pattern-either:
    # ---- Inline forms: query built in the same expression as the call ----
    - pattern: $CUR.execute(f"...{X}...", ...)
    - pattern: $CUR.executemany(f"...{X}...", ...)
    - pattern: $CUR.execute("..." % X, ...)
    - pattern: $CUR.executemany("..." % X, ...)
    - pattern: $CUR.execute("..."format(...), ...)
    - pattern: $CUR.executemany("..."format(...), ...)
    # ---- Variable forms: query assigned then executed in the same scope ----
    - pattern: |
        $Q = f"...{X}..."
        ...
        $CUR.execute($Q, ...)
    - pattern: |
        $Q = "..." % X
        ...
        $CUR.execute($Q, ...)
    - pattern: |
        $Q = "..."format(...)
        ...
        $CUR.execute($Q, ...)
    - pattern: |
        $Q = f"...{X}..."
        ...
        $CUR.executemany($Q, ...)
    - pattern: |
        $Q = "..." % X
        ...
        $CUR.executemany($Q, ...)
    - pattern: |
        $Q = "..."format(...)
        ...
        $CUR.executemany($Q, ...)
  fix: |
    # Replace string formatting with parameterized query, e.g.:
    # cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

3.1 规则要点说明

- 顶层 pattern-either 平铺 12 条分支, 每条都自治, 避免多层 patterns 嵌套时元变量绑定难以预期。

- 前 6 条覆盖 inline 形态(`execute(f"..."` 等),用 `...` 兼容多参数。
- 后 6 条覆盖 变量形态(先赋值给 `$Q`,再 `execute`),元变量 `$Q` 强制赋值左值与 `execute` 实参一致。
- 三个字符串格式化枝(f-string / `%` / `.format`)× 两种 sink(`execute` / `executemany`)= 6 个变量形分支,完整正交。
- metadata 标 CWE-89 与 OWASP A03,便于 CI 报表分组;fix 字段给参数化写法的注释提示,作为规范化建议(不做无脑替换)。

四、Semgrep 运行输出

4.1 命令行(仅加载自写规则,符合扫描约束)

```
cd code-repo
semgrep -c .semgrep/sql-string-format.yml \
  --metrics=off --no-git-ignore --disable-version-check .
```

命令未引入任何官方规则包(p/python、p/owasp-top-ten 等),-c 仅指向自写规则文件。

4.2 主要输出

```
+-----+
| Scan Status |
+-----+
  Scanning 16 files with 1 Code rule:
  Scanning 3 files.

+-----+
| Scan Summary |
+-----+
[OK] Scan completed successfully.
* Findings: 3 (3 blocking)
* Rules run: 1
* Targets scanned: 3
* Parsed lines: ~100.0%
* No ignore information available
Ran 1 rule on 3 files: 3 findings.

+-----+
| 3 Code Findings |
+-----+

db.py
>>> semgrep.python-sqli-string-format-execute
    << Blocking >>
    SQL query is built via string formatting (% , f-string, or .format()) and then
    passed to a DB-API
    execute()/executemany(). This is SQL injection. Use parameterized queries:
        cursor.execute(sql,
        (param,)).

>> Autofix > # Replace string formatting with parameterized query, e.g.: #
    cursor.execute("SELECT * FROM
    users WHERE username = ?", (username,))
39 query = "SELECT * FROM users WHERE username = '%s'" % username
40 cur.execute(query)
: -----
>> Autofix > # Replace string formatting with parameterized query, e.g.: #
    cursor.execute("SELECT * FROM
    users WHERE username = ?", (username,))
50 query = f"SELECT * FROM users WHERE email = '{email}'"
51 cur.execute(query)
: -----
>> Autofix > # Replace string formatting with parameterized query, e.g.: #
    cursor.execute("SELECT * FROM
    users WHERE username = ?", (username,))
61 query = "INSERT INTO users (username, pw_hash, email) VALUES ('{ }', '{ }',
    '{ }')".format(
62     username, pw_hash, email
63 )
```

4.3 命中汇总

#	文件:行	形态	命中代码
1	db.py:39	% 拼接	query = "SELECT * FROM users WHERE username = '%s'" % username
2	db.py:50	f-string	query = f"SELECT * FROM users WHERE email = '{email}'"
3	db.py:61	.format()	query = "INSERT INTO users ... VALUES ('{}', '{}', '{}')".format(...)

五、结论

5.1 命中位置

仅自写规则的扫描产生 3 条命中,全部位于 db.py 的 39、50、61 行,对应三个 DAL 查询函数中三种典型字符串格式化拼接形态(% / f-string / .format()),扫描时长约 1 秒。

5.2 与人工分析的一致性

完全一致。人工分析阶段已确认 db.py 内三个函数存在 SQLi,且明确豁免了 store_token、list_tokens 这两个使用参数化占位符的函数。本次自写规则扫描在仓库内 16 个文件中只对这三处真阳性命中,对参数化的安全调用无误报,精度 3/3。

5.3 仍可能的漏报与误报局限

- 漏报 1(跨函数 / 跨文件):若 SQL 字符串在另一函数中拼接好后,通过参数传入 execute(例如 helper 返回 query),本规则只看同一作用域,无法跟踪跨作用域的污点流。需要使用 Semgrep 的 taint-mode 才能覆盖。
- 漏报 2(其他坏模式):仅识别 % / f-string / .format,对 "a"+x+"b" 这种 + 直接拼接、或经 str.join()、printf-style logging 二次构造的查询会漏掉。
- 漏报 3(异名 sink):仅匹配 execute / executemany。对 SQLAlchemy Core 的 conn.execute(text(query))、Django 的 cursor.callproc()、psycopg2 的 cur.copy_expert() 等等价 sink 不命中。
- 误报风险:当 query 来自完全可信源(例如内部硬编码常量经 .format() 注入表名),规则仍会触发。语义上属于一种「合理告警」,但工程上需要人工 triage 或加 nosemgrep 注释豁免。
- 规则范围声明:本次实验只覆盖 SQL 注入字符串拼接 一类。仓库内还存在 subprocess shell=True 命令注入、yaml.load 反序列化、MD5 密码哈希、Flask debug=True 等问题,均不在本规则的目标范围内,后续可分别撰写规则补齐。

六、提交清单(按要求 5)

编号	材料	本报告中位置 / 仓库路径
1	学号 / 姓名	封面信息表
2	源码级分析流程图	第二节,PNG 已嵌入
3	全部自写规则	第三节,完整列出 .semgrep/sql-string-format.yml
4	Semgrep 运行输出(命令行 + 主要输出)	第四节,/tmp/semgrep_self.txt
5	结论(命中、与人工对比、漏/误报局限)	第五节

完整命令、规则路径、扫描时间戳与日志详见仓库根 `.semgrep/` 目录与 `/tmp/semgrep_self.txt`,均可重复执行复现。